

Fundamental level

Módulo. Entorno y herramientas

- Contenidos clave:
 - Instalación: Python 3.12, Ambientes virtuales / Poetry
 - IDEs: PyCharm y extensiones útiles, VS Code
 - Estructura de proyecto con pyproject.toml / requirements.txt
 - PEP 8 (estilo) y PEP 20 (Zen de Python); automatización con black, isort, ruff y pre-commit
- Objetivos:
 - Instalar/gestionar versiones y entornos
 - Configurar IDE y herramientas de calidad
 - Aplicar PEP 8/PEP 20 y documentar excepciones locales
- Laboratorio:
 - Crear proyecto con Poetry, activar venv, instalar black/isort/ruff
 - Configurar pre-commit; corregir infracciones PEP 8 iniciales

Módulo. Fundamentos del lenguaje

- Contenidos clave:
 - Sintaxis, indentación, variables y alcance
 - Tipos básicos y colecciones (list, dict, set, tuple)
 - Control de flujo (if, switch, for, while) y pattern matching
 - Errores y excepciones (try-except)
 - Control o gestión de excepciones
- Objetivos:
 - Manejar estructuras de datos y control de flujo
 - Implementar manejo de errores robusto
 - Usar pattern matching en casos adecuados y expresiones regulares
- Laboratorio:
 - Script que lee JSON, filtra/agrega datos y maneja errores de archivo/formato

Módulo. Funciones y programación “pythonic”

- Contenidos clave:
 - Funciones, argumentos posicionales/nombrados, *args/**kwargs
 - Lambdas, closures y decoradores
 - Iteradores, generadores, comprensiones
 - Context managers (with)
- Objetivos:
 - Diseñar APIs de funciones claras y expresivas
 - Implementar decoradores y generadores útiles
 - Crear context managers para recursos
- Laboratorio:
 - Decorador de reintentos con backoff
 - Generador por lotes y context manager de temporización

Módulo. Objetos y modelos de datos

- Contenidos clave:
 - Clases, herencia, composición, dunder methods
 - dataclasses y attrs
 - Pydantic para validación/serialización
- Objetivos:
 - Modelar entidades con comportamientos y validaciones
 - Serializar y validar entradas/salidas
- Laboratorio:
 - dataclass Order con cálculos derivados y comparaciones
 - Modelos Pydantic (OrderIn/OrderOut) y conversión a entidad

Módulo. Tipado estático opcional y calidad

- Contenidos clave:
 - Type hints y typing avanzado (Union, Literal, TypedDict, Protocol)
 - mypy/pyright; límites del tipado dinámico
 - PEP 8 aplicado con ruff/black/isort; PEP 20 como guía de diseño

- Pre-commit y checks en CI
- Objetivos:
 - Anotar tipos y verificar estáticamente
 - Hacer cumplir PEP 8 y documentar excepciones
 - Integrar linters/formatters en CI
- Laboratorio:
 - Anotar tipos en el código previo, ejecutar mypy y ruff; configurar pre-commit

Módulo. Librería estándar y E/S

- Contenidos clave:
 - pathlib y manejo de archivos
 - CSV/JSON/YAML: parseo y serialización
 - datetime y zonas horarias
 - logging y configuración
 - subprocess y automatización
- Objetivos:
 - Manipular archivos/rutas de forma segura
 - Configurar logging estructurado
- Laboratorio:
 - Ingesta de CSV, métricas y exportación a JSON; logging con distintos niveles

Módulo. HTTP y consumo de APIs (Smocker)

- Contenidos clave:
 - requests, aiohttp y httpx (http2)
 - Timeouts, reintentos, manejo de errores
 - Streaming de respuestas y uso eficiente de memoria
- Objetivos:
 - Construir clientes HTTP robustos
 - Gestionar resiliencia y errores
- Laboratorio:
 - Cliente httpx con reintentos/timeouts; descarga por streaming a disco

Intermediate level

Módulo. Acceso a datos y ORM

- Contenidos clave:
 - sqlite3 y drivers para PostgreSQL/SQL Server
 - SQLAlchemy Core y ORM; relaciones y consultas
 - Migraciones con Alembic
 - Introducción a MongoDB (Motor)
- Objetivos:
 - Modelar entidades y relaciones en ORM
 - Gestionar migraciones y transacciones
- Laboratorio:
 - Modelos User/Order/OrderItem, CRUD básico, migración Alembic y pruebas en SQLite en memoria

Módulo. APIs web con FastAPI (Automatización)

- Contenidos clave:
 - Estructura de proyecto, routers y dependencias
 - Esquemas Pydantic, validación y OpenAPI
 - Autenticación JWT, middlewares, CORS
 - Testing de endpoints con pytest + aiohttp
- Objetivos:
 - Exponer una API coherente y validada
 - Asegurar endpoints y documentarlos
- Laboratorio:
 - CRUD de Orders con validación; login JWT básico; tests de integración con DB temporal

Módulo. Pruebas y TDD

- Contenidos clave:
 - pytest: fixtures, parametrización, markers
 - Mocking con unittest.mock
 - Property-based testing con Hypothesis

- Cobertura e integración en CI
- Objetivos:
 - Practicar TDD y diseñar pruebas confiables
 - Asegurar cobertura suficiente y suite estable
- Laboratorio:
 - Implementar una historia nueva con TDD; añadir test de propiedades; reporte de cobertura

Módulo. Concurrencia y rendimiento

- Contenidos clave:
 - GIL y sus implicaciones
 - threading y concurrent.futures
 - asyncio: event loop, async/await
 - multiprocessing para CPU-bound
 - Medición con timeit y cProfile
- Objetivos:
 - Elegir el modelo de concurrencia apropiado
 - Implementar E/S concurrente y medir mejoras
- Laboratorio:
 - Fetcher concurrente con httpx.AsyncClient y semáforo; comparación con versión síncrona; pequeño cálculo CPU-bound con ProcessPoolExecutor

Módulo. Principios SOLID aplicados en Python

- Contenidos clave:
 - SRP, OCP, LSP, ISP, DIP en clave “pythonic”
 - Inversión de dependencias con Protocols, factories y provider patterns
 - Acoplamiento, cohesión y testabilidad
- Objetivos:
 - Aplicar SOLID a servicios y dominio
 - Reducir acoplamiento y mejorar extensibilidad
- Laboratorio:
 - Refactor de servicio para depender de puerto (Protocol); implementaciones memoria/SQL y verificación LSP

Módulo. Patrones de diseño

- Contenidos clave:
 - Creacionales: Factory, Abstract Factory, Builder, Singleton (cuándo evitarlo)
 - Estructurales: Adapter, Facade, Composite, Decorator, Proxy
 - Comportamiento: Strategy, Observer, Command, Mediator, Template Method, State
 - Patrones idiomáticos: decoradores, context managers, dataclasses
- Objetivos:
 - Implementar patrones relevantes con ejemplos reales
 - Identificar antipatrones y señales de refactor
- Laboratorio:
 - Strategy para precios; Decorator de caché; Adapter para proveedor externo; pruebas con pytest

Módulo. Ciencia de datos

- Contenidos clave:
 - NumPy, Polars y Pandas
 - scikit-learn: modelos clásicos
 - Serialización de modelos e inferencia básica
- Objetivos:
 - Manipular datos y entrenar un modelo simple
 - Exponer inferencia mínima
- Laboratorio:
 - Cargar/limpiar CSV en Pandas; entrenar un clasificador; guardar con joblib y probar inferencia

Módulo. Arquitectura Hexagonal (Puertos y Adaptadores)

- Contenidos clave:
 - Capas: dominio, aplicación e infraestructura
 - Puertos (interfaces/Protocols) y adaptadores (SQL, HTTP, mensajería)
 - Casos de uso y orquestación; DTOs vs entidades
 - Inyección de dependencias y wiring en FastAPI
 - Pruebas de dominio, contrato y end-to-end
- Objetivos:

- Separar reglas de negocio de detalles de infraestructura
- Definir puertos estables y adaptadores intercambiables
- Laboratorio:
 - Caso de uso CreateOrder con puerto de repositorio y adaptadores en memoria/SQLAlchemy; adaptador de notificación HTTP simulado; pruebas de contrato

Advance level

Módulo. Arquitectura Limpia

- Contenidos clave:
 - Entidades, casos de uso, controladores/presenters/gateways
 - Reglas de dependencia y separación de capas
 - Unit of Work y eventos de dominio
 - Estrategias de migración hacia arquitectura limpia
- Objetivos:
 - Estructurar un servicio con capas independientes y reglas claras
 - Gestionar transacciones y publicar eventos
- Laboratorio:
 - Reestructurar Orders a arquitectura limpia; introducir UoW y Presenter; evento OrderCreated manejado en aplicación

Módulo. Empaquetado, distribución y CI/CD

- Contenidos clave:
 - Construcción de ruedas (wheels) y publicación (PyPI/artefactos internos)
 - Docker multistage y buenas prácticas
 - Pipelines en GitHub Actions/Azure DevOps
 - Despliegue en Azure (App Service, Container Apps, Functions)
- Objetivos:
 - Empaquetar, contenerizar y automatizar pipelines
 - Publicar artefactos reproducibles
- Laboratorio:
 - Generar wheel; Dockerfile multistage para FastAPI; workflow CI (lint, type-check, tests, build) y push a registry

Módulo. CLI y automatización

- Contenidos clave:
 - argparse, click y Typer
 - Configuración por variables de entorno

- Scripts de mantenimiento
- Objetivos:
 - Construir CLIs productivas y mantenibles
 - Integrar automatizaciones del proyecto
- Laboratorio:
 - CLI con Typer para gestionar Orders (listar/crear/borrar) consumiendo la API; entry point en el paquete

Módulo. Seguridad y mantenimiento

- Contenidos clave:
 - Gestión de secretos y configuración
 - Auditoría de dependencias: pip-audit, safety
 - Actualizaciones (PEP 440) y compatibilidad
 - Hardening de contenedores y del runtime
- Objetivos:
 - Proteger secretos y reducir superficie de riesgo
 - Mantener dependencias seguras y actualizadas
- Laboratorio:
 - Integrar pydantic-settings; ejecutar pip-audit/safety y resolver hallazgos; Docker sin root y permisos mínimos

Módulo 18. Interoperabilidad y ecosistema mixto (opcional)

- Contenidos clave:
 - Contratos neutrales: OpenAPI y Protobuf
 - Servicios/clients gRPC en Python
 - Mensajería con RabbitMQ/Redis/Kafka
 - Serialización: JSON/Avro/Protobuf
- Objetivos:
 - Definir contratos y comunicarse entre servicios heterogéneos
 - Integrar mensajería para eventos
- Laboratorio:

- Definir .proto de Orders, generar stubs, servidor gRPC y cliente; publicar evento OrderCreated en RabbitMQ/Redis

Proyecto final integrador (Arquitectura Hexagonal/Limpia)

- Objetivos:

- Construir un servicio de Orders con dominio, casos de uso, puertos y adaptadores
- Exponer API FastAPI segura y documentada
- Asegurar calidad (pruebas, lint, tipado) y delivery (Docker, CI/CD)

- Entregables:

- Código con capas (dominio, aplicación, infraestructura) y API
- Pruebas unitarias, de contrato, integración/E2E; migraciones Alembic
- Dockerfile multistage y pipeline CI; README y diagramas
- Evidencia de auditoría de dependencias
- Evaluación: según la rúbrica detallada entregada (arquitectura, dominio, puertos/adaptadores, API, pruebas, calidad, CI/CD, seguridad, observabilidad, rendimiento y documentación)